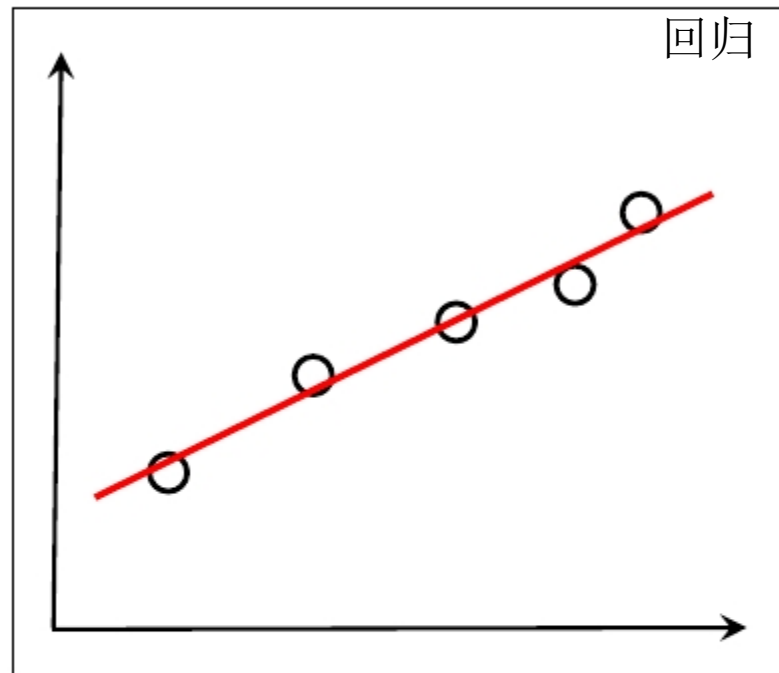
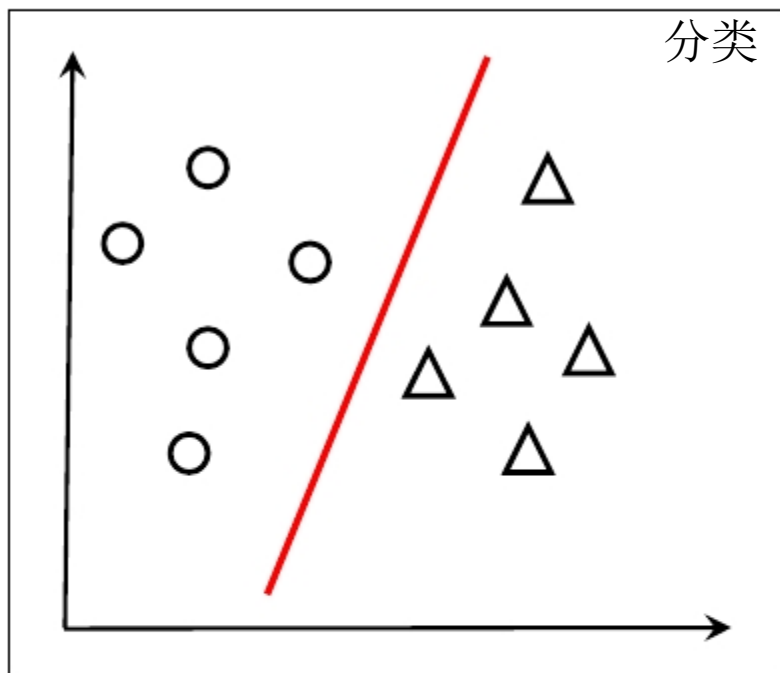


三、线性模型

温铁祥
健康与环境工程学院

线性模型



线性模型(linear model)试图学得一个通过属性的线性组合来进行预测的函数

$$f(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

$$f(x) = w^T x + b$$

简单、基本、可理解性好

线性回归 (linear regression)

□ 离散属性的数值编码

色泽	根蒂	敲声	好瓜
青绿	蜷缩	浊响	是
乌黑	蜷缩	浊响	是
青绿	硬挺	清脆	否
乌黑	稍蜷	沉闷	否



色泽	根蒂	敲声	好瓜	
2	3	2	1	0
3	3	2	1	0
2	1	1	0	1
3	2	3	0	1

□ 若属性间存在“序”（order）关系：OrdinalEncoder

➤ 通过连续化将其转化为连续值

➤ 例如：预测{婴儿、儿童、青少年、中年人、老年人}，则可用 {0, 1, 2, 3, 4} 表示对应类别。

□ 若属性间不存在序关系

➤ 独热编码： OneHotEncoder ， 1-of-K

➤ 例如：预测{猫、鸡、狗}，则

(1, 0, 0) ↔ 猫， (0, 1, 0) ↔ 鸡， (0, 0, 1) ↔ 狗

线性回归 (linear regression)

$$f(x) = wx_i + b \quad \text{使得} \quad f(x_i) \simeq y_i$$

令均方误差最小化, 有 $(w^*, b^*) = \arg \min_{(w, b)} \sum_{i=1}^m (f(x_i) - y_i)^2$

$$= \arg \min_{(w, b)} \sum_{i=1}^m (y_i - wx_i - b)^2$$

对 $E_{(w, b)} = \sum_{i=1}^m (y_i - wx_i - b)^2$ 进行最小二乘参数估计

线性回归

分别对 w 和 b 求导：

$$\frac{\partial E_{(w,b)}}{\partial w} = 2 \left(w \sum_{i=1}^m x_i^2 - \sum_{i=1}^m (y_i - b) x_i \right)$$
$$\frac{\partial E_{(w,b)}}{\partial b} = 2 \left(mb - \sum_{i=1}^m (y_i - wx_i) \right)$$

令导数为 0, 得到闭式解(closed-form solution)：

$$w = \frac{\sum_{i=1}^m y_i (x_i - \bar{x})}{\sum_{i=1}^m x_i^2 - \frac{1}{m} \left(\sum_{i=1}^m x_i \right)^2} \quad b = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i)$$

多元(multi-variate)线性回归

$$f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b \quad \text{使得} \quad f(\mathbf{x}_i) \simeq y_i$$

$$\mathbf{x}_i = (x_{i1}; x_{i2}; \dots; x_{id}) \quad y_i \in \mathbb{R}$$

把 $\mathbf{w}$ 和 b 吸收入向量形式 $\hat{\mathbf{w}} = (\mathbf{w}; b)$ ，数据集表示为

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^T & 1 \\ \mathbf{x}_2^T & 1 \\ \vdots & \vdots \\ \mathbf{x}_m^T & 1 \end{pmatrix} \quad \mathbf{y} = (y_1; y_2; \dots; y_m)$$

多元线性回归

同样采用最小二乘法求解（Least squares method），有

$$\hat{\boldsymbol{w}}^* = \arg \min_{\hat{\boldsymbol{w}}} (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}})^T (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}})$$

令 $E_{\hat{\boldsymbol{w}}} = (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}})^T (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}})$ ，对 $\hat{\boldsymbol{w}}$ 求导：

$$\frac{\partial E_{\hat{\boldsymbol{w}}}}{\partial \hat{\boldsymbol{w}}} = 2\boldsymbol{X}^T (\boldsymbol{X}\hat{\boldsymbol{w}} - \boldsymbol{y}) \quad \text{令其为零可得 } \hat{\boldsymbol{w}}$$

然而，麻烦来了：涉及矩阵求逆！

□ 若 $\boldsymbol{X}^T \boldsymbol{X}$ 满秩或正定，则 $\hat{\boldsymbol{w}}^* = (\boldsymbol{X}^T \boldsymbol{X})^{-1} \boldsymbol{X}^T \boldsymbol{y}$

□ 若 $\boldsymbol{X}^T \boldsymbol{X}$ 不满秩，则可解出多个 $\hat{\boldsymbol{w}}$

此时需求助于归纳偏好，或引入正则化 (regularization)

多元线性回归

□ 线性拟合实例

```
import numpy as np
```

```
X = 2 * np.random.rand(100, 1)
```

```
y = 4 + 3 * X + np.random.randn(100, 1)
```

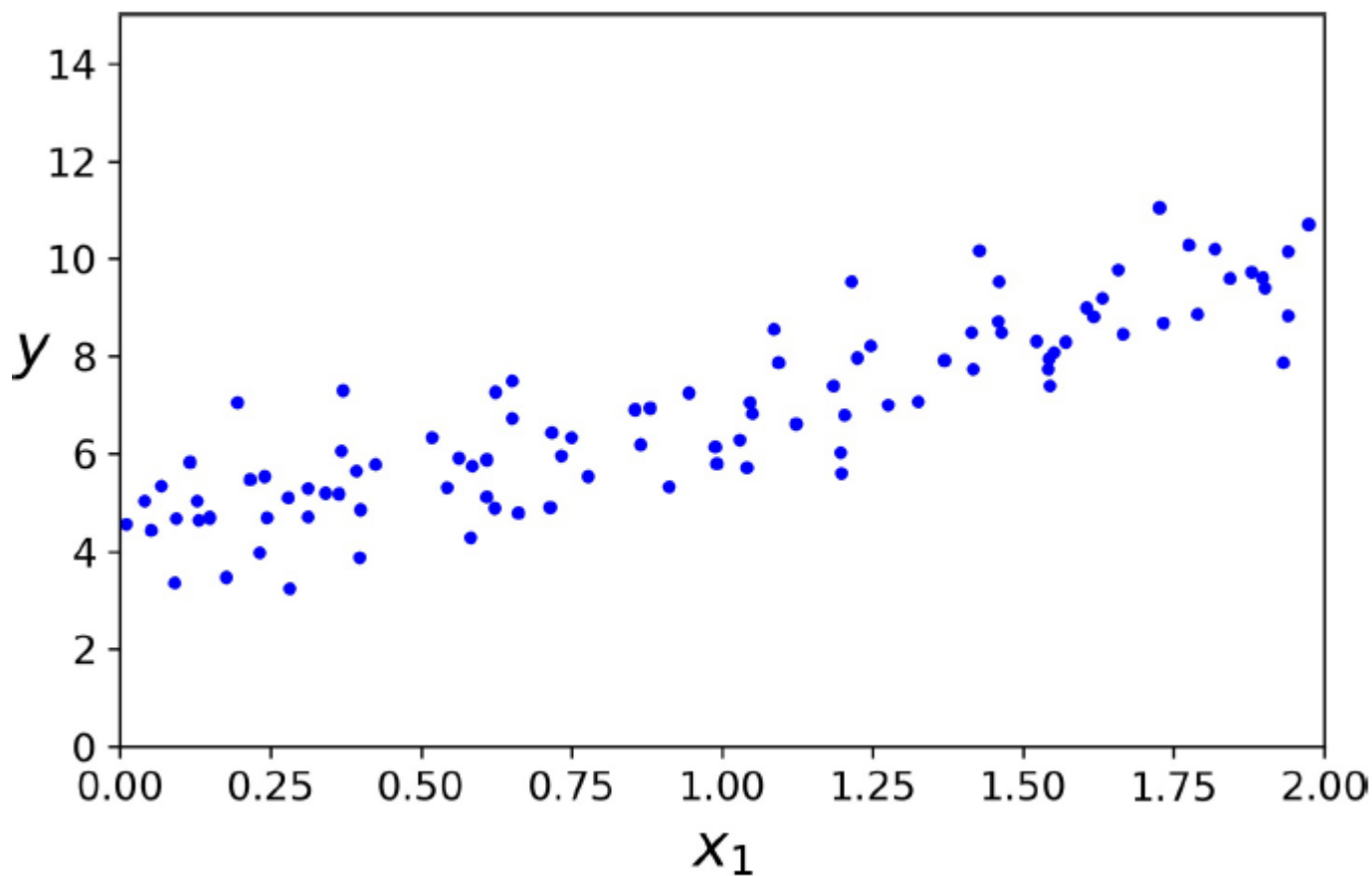


Figure 4-1. Randomly generated linear dataset

多元线性回归

□ 线性拟合实例

```
X_b = np.c_[np.ones((100, 1)), X] # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)

>>> theta_best
array([[4.21509616],
       [2.77011339]])

>>> X_new = np.array([[0], [2]])
>>> X_new_b = np.c_[np.ones((2, 1)), X_new] # add x0 = 1 to each instance
>>> y_predict = X_new_b.dot(theta_best)
>>> y_predict
array([[4.21509616],
       [9.75532293]])
```

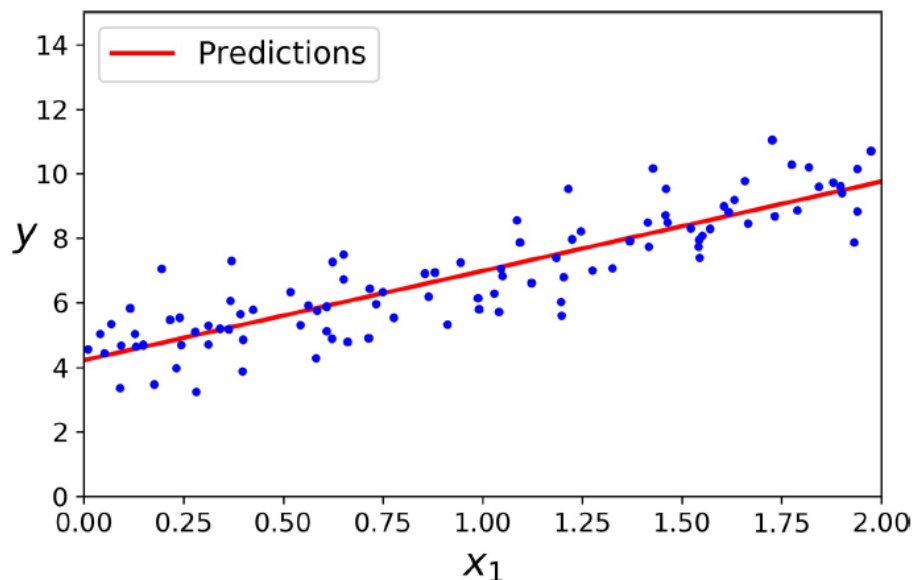


Figure 4-2. Linear Regression model predictions

多元线性回归

□ 线性回归模型的最小二乘解：

$$f(\hat{\mathbf{x}}_i) = \hat{\mathbf{x}}_i^{\mathrm{T}} (\mathbf{X}^{\mathrm{T}} \mathbf{X})^{-1} \mathbf{X}^{\mathrm{T}} \mathbf{y} . \quad (3.12)$$

- 计算复杂度通常为 $O(n^{2.4})$ 到 $O(n^3)$
- 闭式解涉及矩阵求逆，逆不一定存在！

多元线性回归

□ 梯度下降法 (Gradient descent method)

- 一种通用的**优化算法**
- 基本思想：迭代地调整参数从而使成本函数最小化

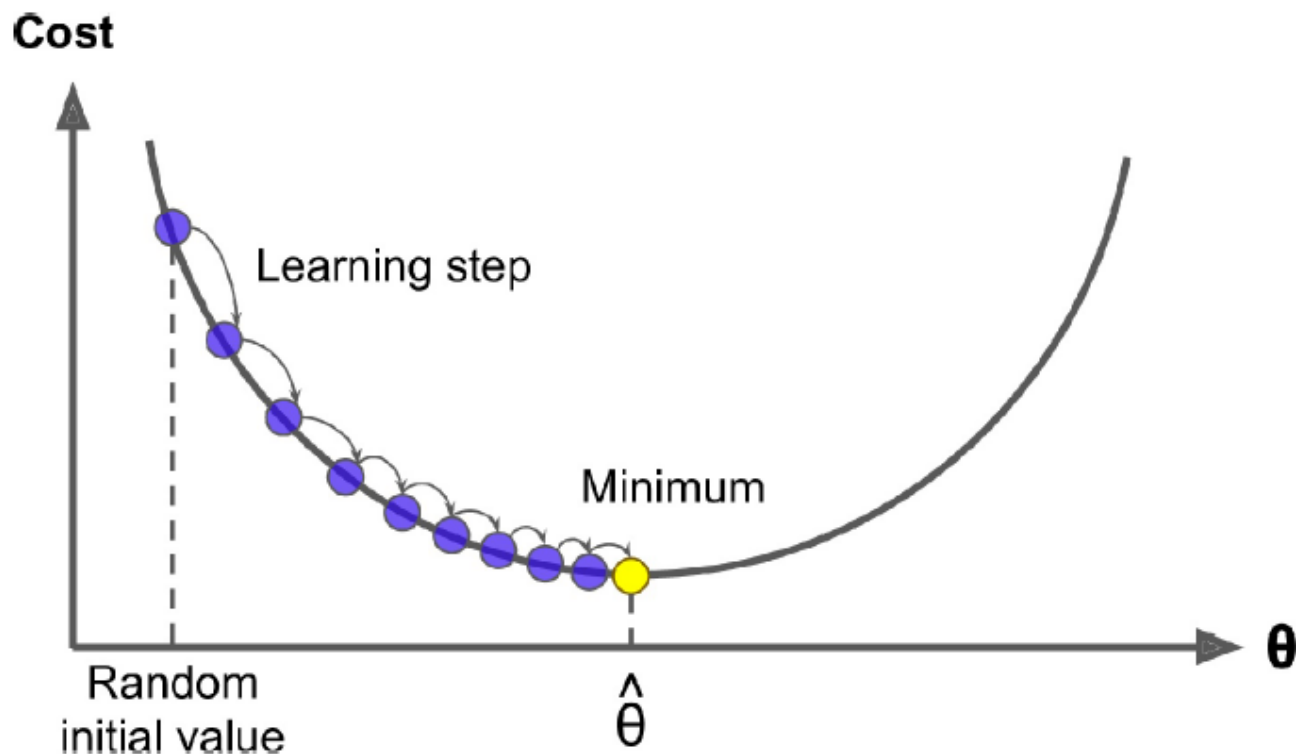


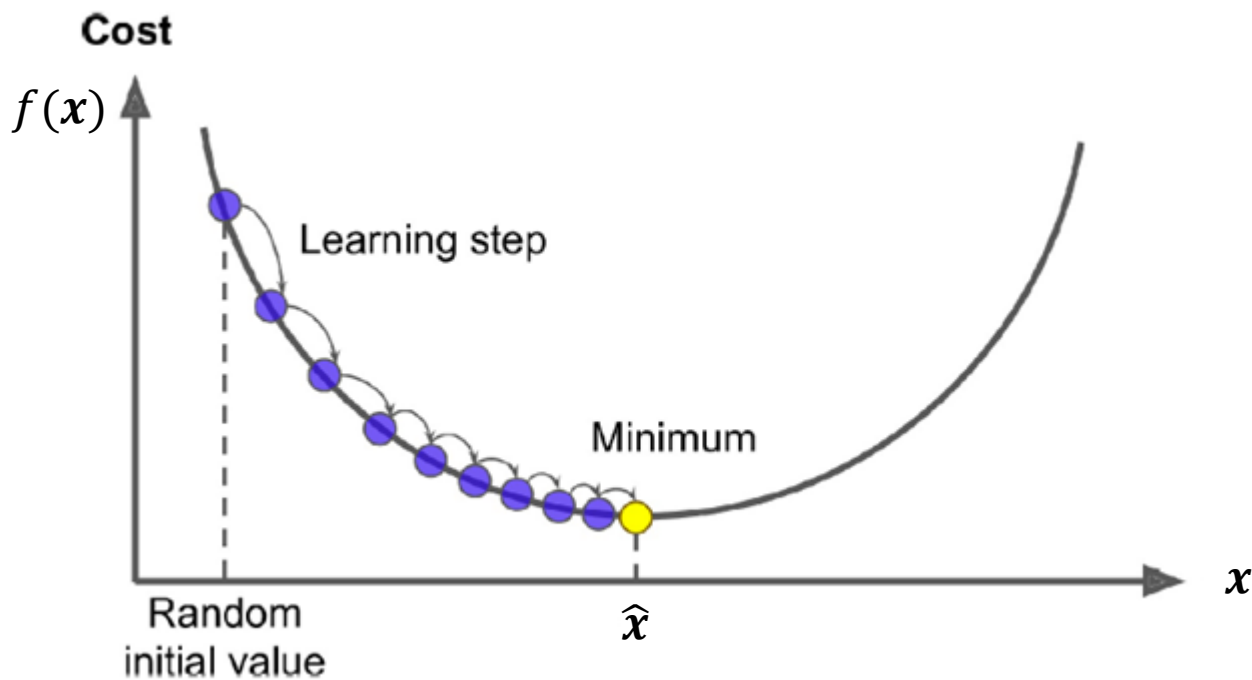
Figure 4-3. In this depiction of Gradient Descent, the model parameters are initialized randomly and get tweaked repeatedly to minimize the cost function; the learning step size is proportional to the slope of the cost function, so the steps gradually get smaller as the parameters approach the minimum

多元线性回归

□ 梯度下降法

cost function, loss function

- 考虑无约束优化问题 $\min_{\mathbf{x}} f(\mathbf{x})$ ，其中 $f(\mathbf{x})$ 为连续可微函数。
若能构造一个序列 $\mathbf{x}^1, \mathbf{x}^2, \mathbf{x}^3, \dots$ 满足 $f(\mathbf{x}^{t+1}) < f(\mathbf{x}^t)$, $t = 0, 1, 2, \dots$
则不断执行该过程即可收敛到局部极小点。



多元线性回归

□ 梯度下降法

- 根据泰勒展式有

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \Delta\mathbf{x}^T \cdot \nabla f(\mathbf{x})$$

- 于是，欲满足 $f(\mathbf{x} + \Delta\mathbf{x}) < f(\mathbf{x})$ ，可选择

$$\Delta\mathbf{x} = -\eta \nabla f(\mathbf{x})$$

- 其中步长 $\eta > 0$ 。则有：

$$f(\mathbf{x} + \Delta\mathbf{x}) \cong f(\mathbf{x}) - \eta (\nabla f(\mathbf{x}))^T \cdot \nabla f(\mathbf{x}) = f(\mathbf{x}) - \eta \|\nabla f(\mathbf{x})\|_2^2$$

$$f(\mathbf{x} + \Delta\mathbf{x}) \leq f(\mathbf{x})$$

- 因此：

$$\mathbf{x}^{t+1} = \mathbf{x}^t + \Delta\mathbf{x} = \mathbf{x}^t - \eta \nabla f(\mathbf{x}) \quad t = 0, 1, 2, \dots$$

$$f(\mathbf{x}^0) > f(\mathbf{x}^1) > f(\mathbf{x}^2) > \dots > f(\mathbf{x}^t) > f(\mathbf{x}^{t+1}) > \dots$$

多元线性回归

□ 梯度下降法

- 基本思想：迭代地调整参数从而使损失函数 $E(\mathbf{w})$ 最小化
- 搜索方向： $\nabla E(\mathbf{w})$
- 搜索步长： η （学习率，Learning Rate）

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \cdot \nabla E(\mathbf{w})$$

【梯度下降法（伪代码）】：

给定训练数据集 $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ ，假设损失函数 $E(\mathbf{w})$ ，求解 $\mathbf{w}$ 。

- 初始化：参数 $\mathbf{w}^0$ ，学习率 η ，迭代次数 n
- for t in range(n)
 - 1) 计算当前损失函数的梯度值： $\nabla E(\mathbf{w}^t)$
 - 2) 更新模型参数： $\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla E(\mathbf{w}^t)$
- return $\mathbf{w}^t$

多元线性回归

□ 线性模型的梯度下降求解

- 沿着损失函数的梯度下降方向搜索！

$$\hat{\omega}^{(t+1)} = \hat{\omega}^{(t)} - \eta \frac{\partial E_{\hat{\omega}}}{\partial \hat{\omega}}$$



$$\frac{\partial E_{\hat{\omega}}}{\partial \hat{\omega}} = 2\mathbf{X}^T (\mathbf{X}\hat{\omega} - \mathbf{y})$$

```
eta = 0.1 # learning rate
n_iterations = 1000
m = 100
```

```
theta = np.random.randn(2,1) # random initialization
```

```
for iteration in range(n_iterations):
    gradients = 2/m * X_b.T.dot(X_b.dot(theta) - y)
    theta = theta - eta * gradients
```

```
>>> theta
array([[4.21509616],
       [2.77011339]])
```

多元线性回归

□ 梯度下降法

- 超参数（hyperparameter）：学习率 η

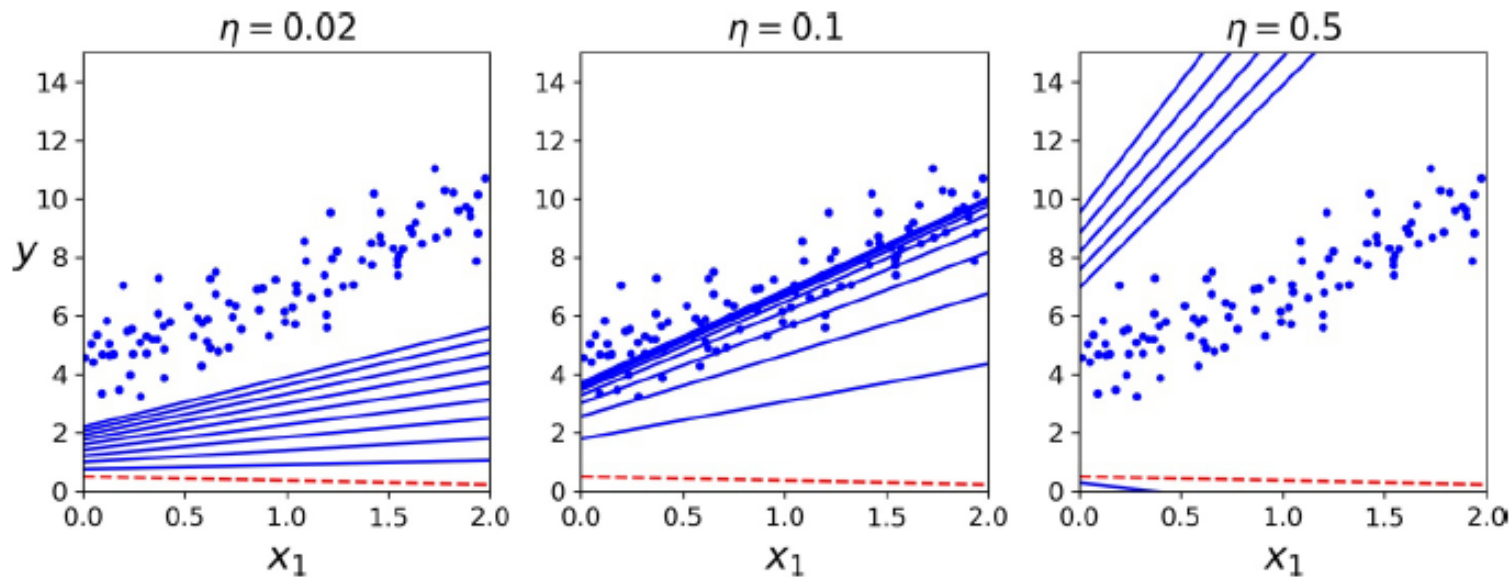


Figure 4-8. Gradient Descent with various learning rates

多元线性回归

□ 梯度下降法

- 超参数（hyperparameter）：学习率 η
- 如果设置太低，需要大量迭代才能收敛，耗时。

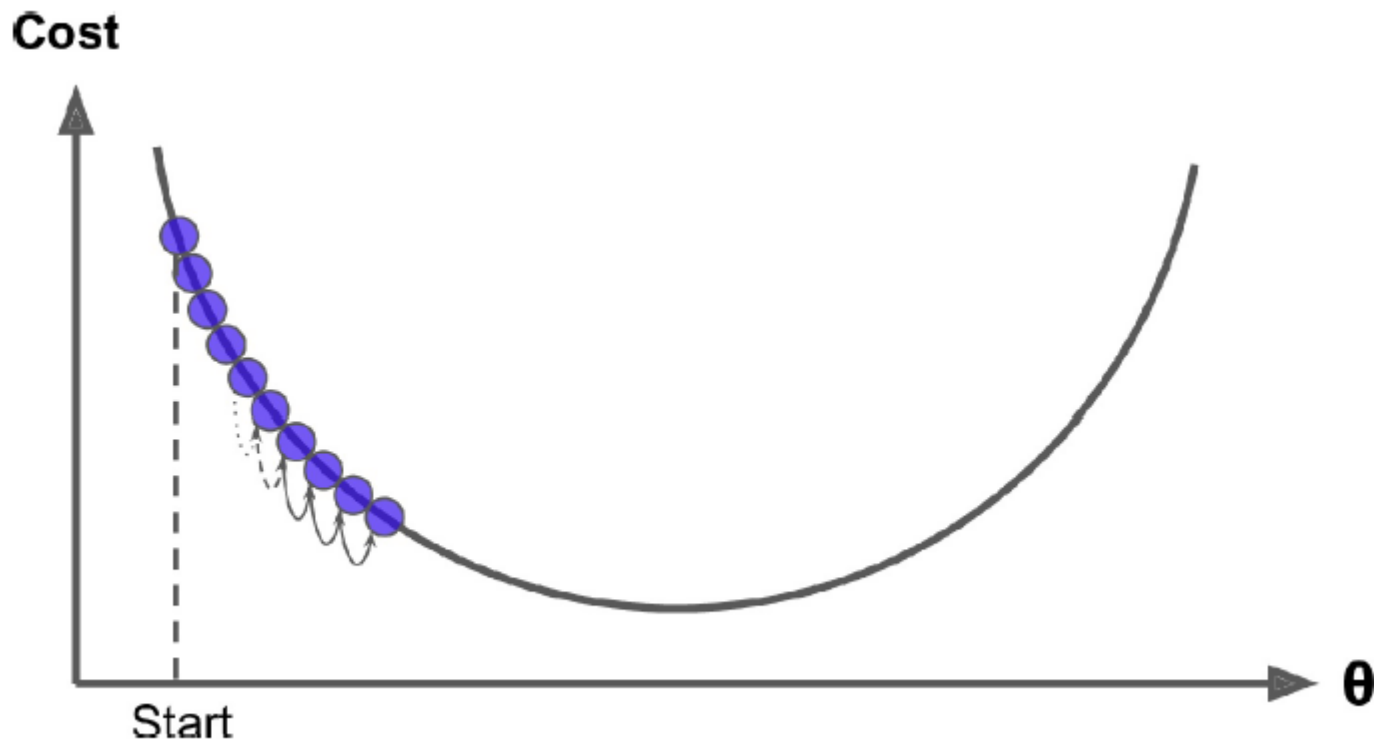


Figure 4-4. The learning rate is too small

多元线性回归

□ 梯度下降法

- 超参数（hyperparameter）：学习率 η
- 如果设置过高，容易导致算法发散。

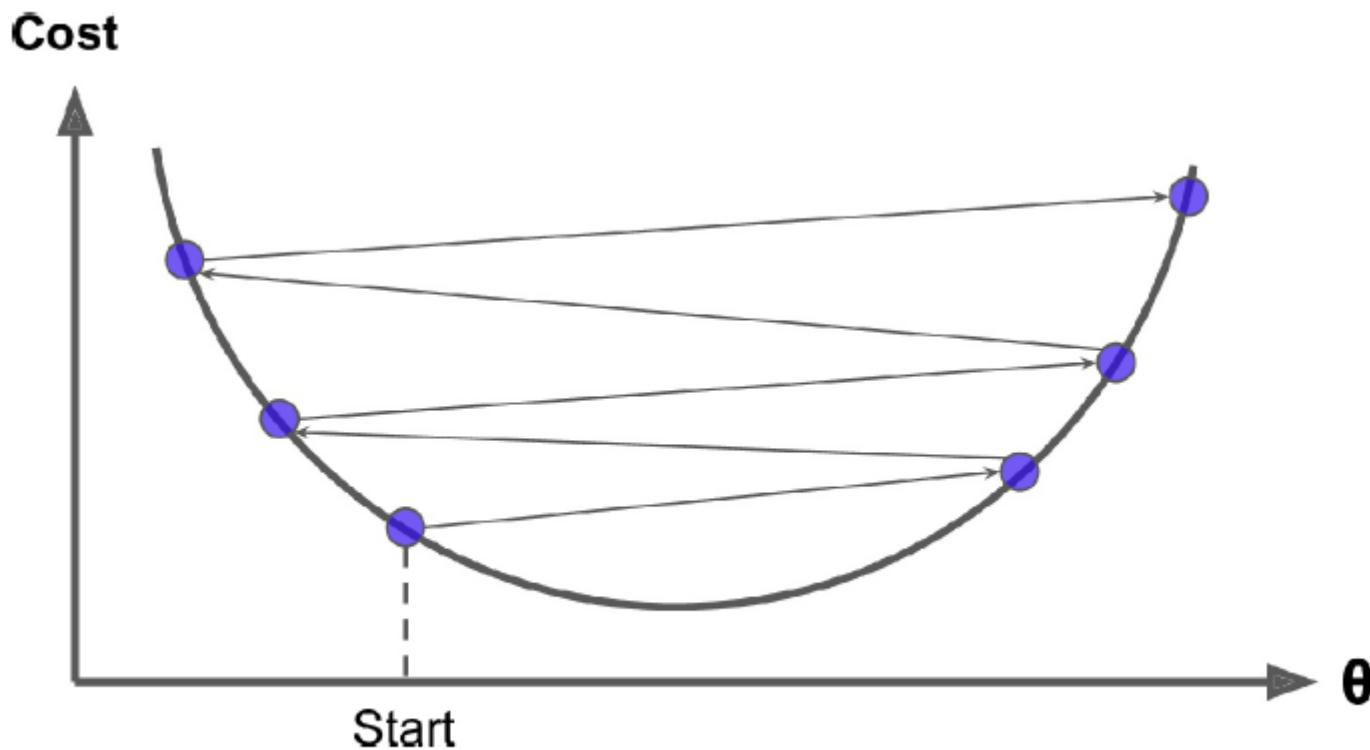


Figure 4-5. The learning rate is too large

多元线性回归

□ 梯度下降法

- 学习率调整策略： e.g. 模拟退火。

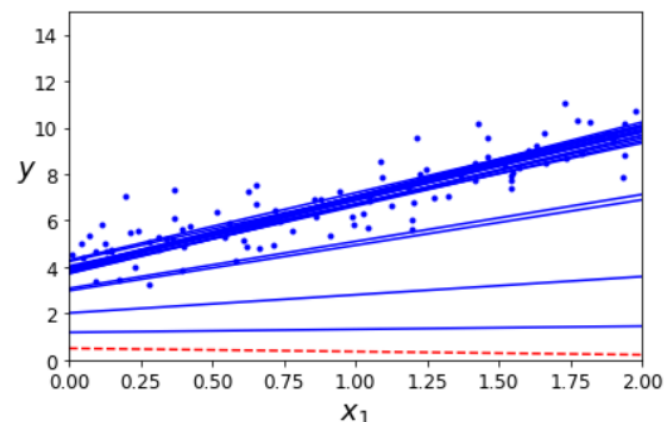
$$e.g. \eta(t) = \frac{t_0}{t + t_1}$$

```
n_epochs = 50
t0, t1 = 5, 50 # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

theta = np.random.randn(2,1) # random initialization

for epoch in range(n_epochs):
    for i in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index:random_index+1]
        yi = y[random_index:random_index+1]
        gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
        eta = learning_schedule(epoch * m + i)
        theta = theta - eta * gradients
```



多元线性回归

□ 梯度下降法

- 对**非凸函数**：可能收敛到**局部**最小值。

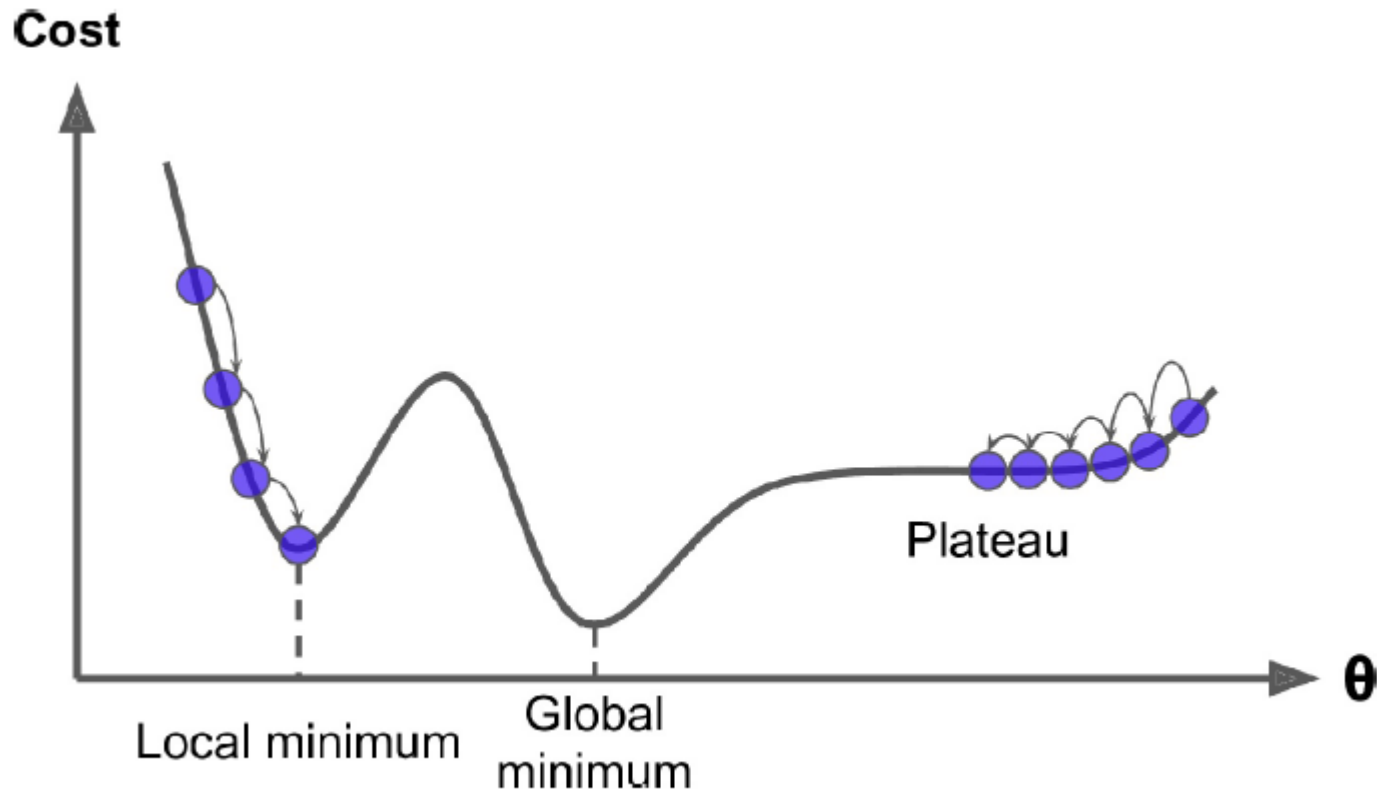


Figure 4-6. Gradient Descent pitfalls

多元线性回归

□ 梯度下降法

- 需要确保所有特征值的大小比例都差不多，否则收敛很慢
- 需要做特征的归一化：

StandardScaler:
$$x' = \frac{x - \bar{x}}{\sigma_x}$$

MinMaxScaler:
$$x' = x'_{min} + \frac{x - x_{min}}{x_{max} - x_{min}} \times (x'_{max} - x'_{min})$$

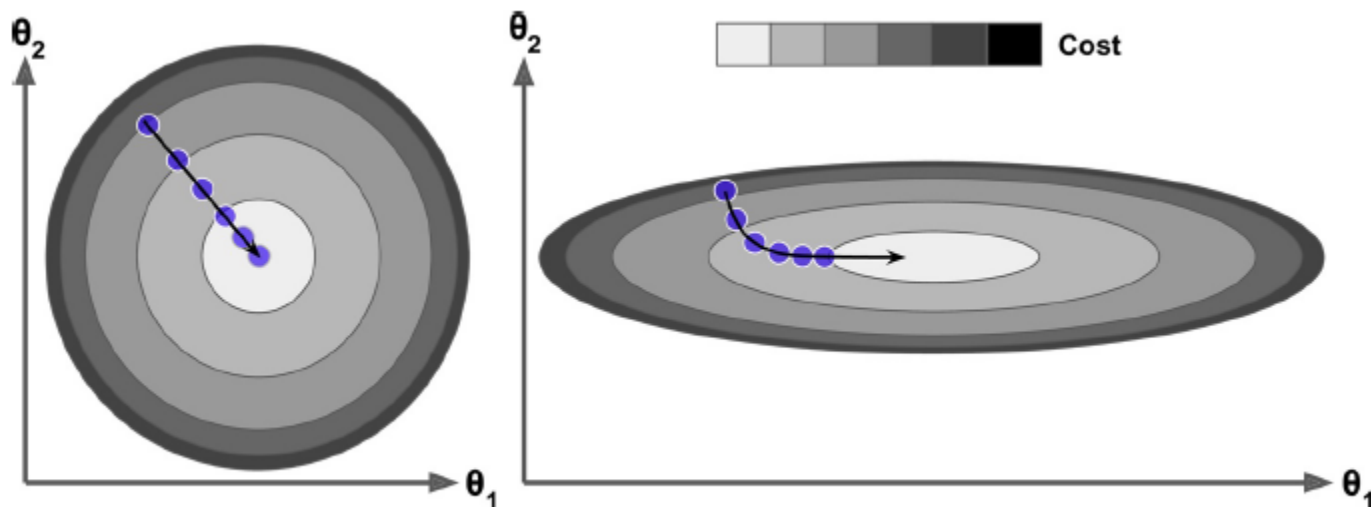


Figure 4-7. Gradient Descent with (left) and without (right) feature scaling

多元线性回归

▣ 梯度下降法

- 随机梯度下降（Stochastic Gradient Descent）

$$\frac{\partial}{\partial \theta} \text{MSE}(\theta) = 2(x_i^T \theta - y_i)x_i$$

- 批量梯度下降（Batch Gradient Descent）

$$\nabla_{\theta} \text{MSE}(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\theta) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$$

- 小批量梯度下降（Mini-batch Gradient Descent）

多元线性回归

□ 梯度下降法

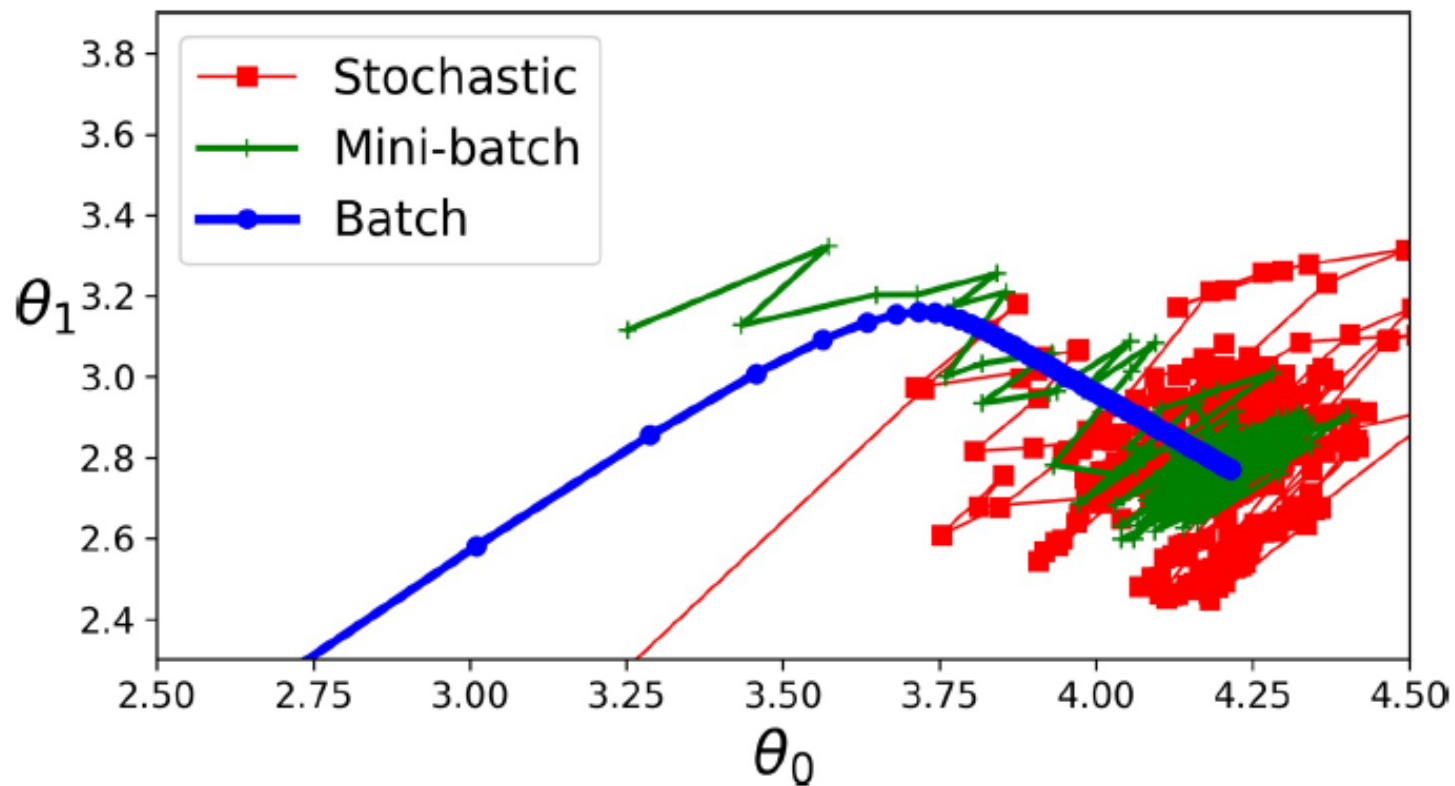


Figure 4-11. Gradient Descent paths in parameter space

多元线性回归

□ 梯度下降法

- 求解无约束问题最简单、最经典的方法之一
- 可用于训练除线性回归之外的许多其他模型
- 若目标函数为凸函数时，局部极小点就对应函数的全局最小点
- 一种常用的一阶（**first-order**）优化方法
- 若目标函数二阶连续可微时，可用更精确的二阶泰勒展开
- 牛顿法就是经典的二阶方法，但涉及Hessian矩阵求逆，计算复杂度高

多元线性回归

□ 线性回归的Scikit-Learn实现

- 闭式方程求解（closed-form solution）
- 梯度下降法求解（gradient descent method）

算法	m 很大	核外支持	n 很大	超参数	要求缩放	Scikit-Learn
标准方程	快	否	慢	0	否	N/A
SVD	快	否	慢	0	否	LinearRegression
批量 GD	慢	否	快	2	是	SGDRegressor
随机 GD	快	是	快	≥ 2	是	SGDRegressor
小批量 GD	快	是	快	≥ 2	是	SGDRegressor

m: 样本的个数

n: 特征的个数

多项式回归 (polynomial regression)

□ 多项式回归模型

- 基本思想：用多项式基函数去拟合两个变量之间的线性关系。

The polynomial regression model

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_m x_i^m + \varepsilon_i \quad (i = 1, 2, \dots, n)$$

can be expressed in matrix form in terms of a design matrix $\mathbf{X}$, a response vector $\vec{y}$, a parameter vector $\vec{\beta}$, and a vector $\vec{\varepsilon}$ of random errors. The i -th row of $\mathbf{X}$ and $\vec{y}$ will contain the x and y value for the i -th data sample. Then the model can be written as a system of linear equations:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ 1 & x_3 & x_3^2 & \cdots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \vdots \\ \varepsilon_n \end{bmatrix},$$

which when using pure matrix notation is written as

$$\vec{y} = \mathbf{X}\vec{\beta} + \vec{\varepsilon}.$$

The vector of estimated polynomial regression coefficients (using [ordinary least squares estimation](#)) is

$$\hat{\vec{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \vec{y},$$

assuming $m < n$ which is required for the matrix to be invertible; then since $\mathbf{X}$ is a [Vandermonde matrix](#), the invertibility condition is guaranteed to hold if all the x_i values are distinct. This is the unique least-squares solution.

Polynomial regression is one example of regression analysis using [basis functions](#) to model a functional relationship between two quantities.

More specifically, it replaces $x \in \mathbb{R}^{d_x}$ in linear regression with polynomial basis $\varphi(x) \in \mathbb{R}^{d_\varphi}$, e.g. $[1, x] \xrightarrow{\varphi} [1, x, x^2, \dots, x^d]$. A drawback of

多项式回归 (polynomial regression)

□ 多项式拟合实例

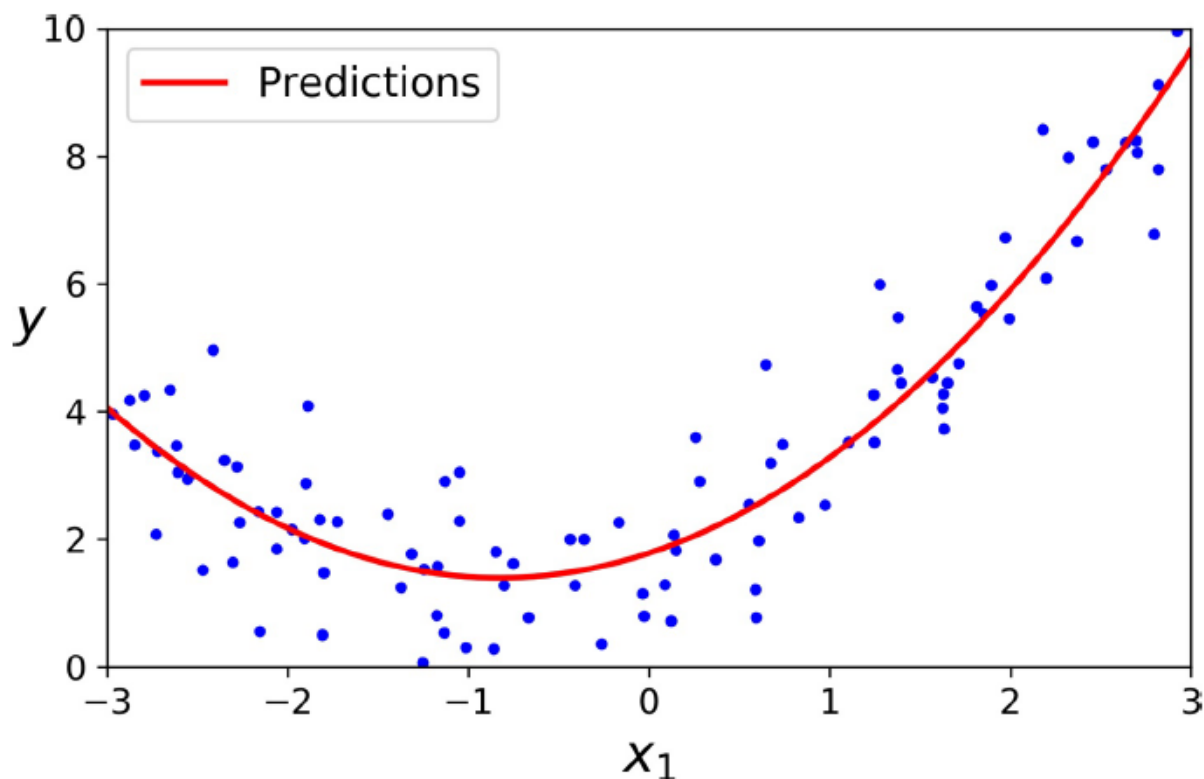


Figure 4-13. Polynomial Regression model predictions

Not bad: the model estimates $\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78$ when in fact the original function was $y = 0.5x_1^2 + 1.0x_1 + 2.0 + \text{Gaussian noise}$.

多项式回归 (polynomial regression)

□ 多项式拟合实例

- ✓ 模型的超参数：多项式的阶次
- ✓ 欠拟合 vs. 过拟合

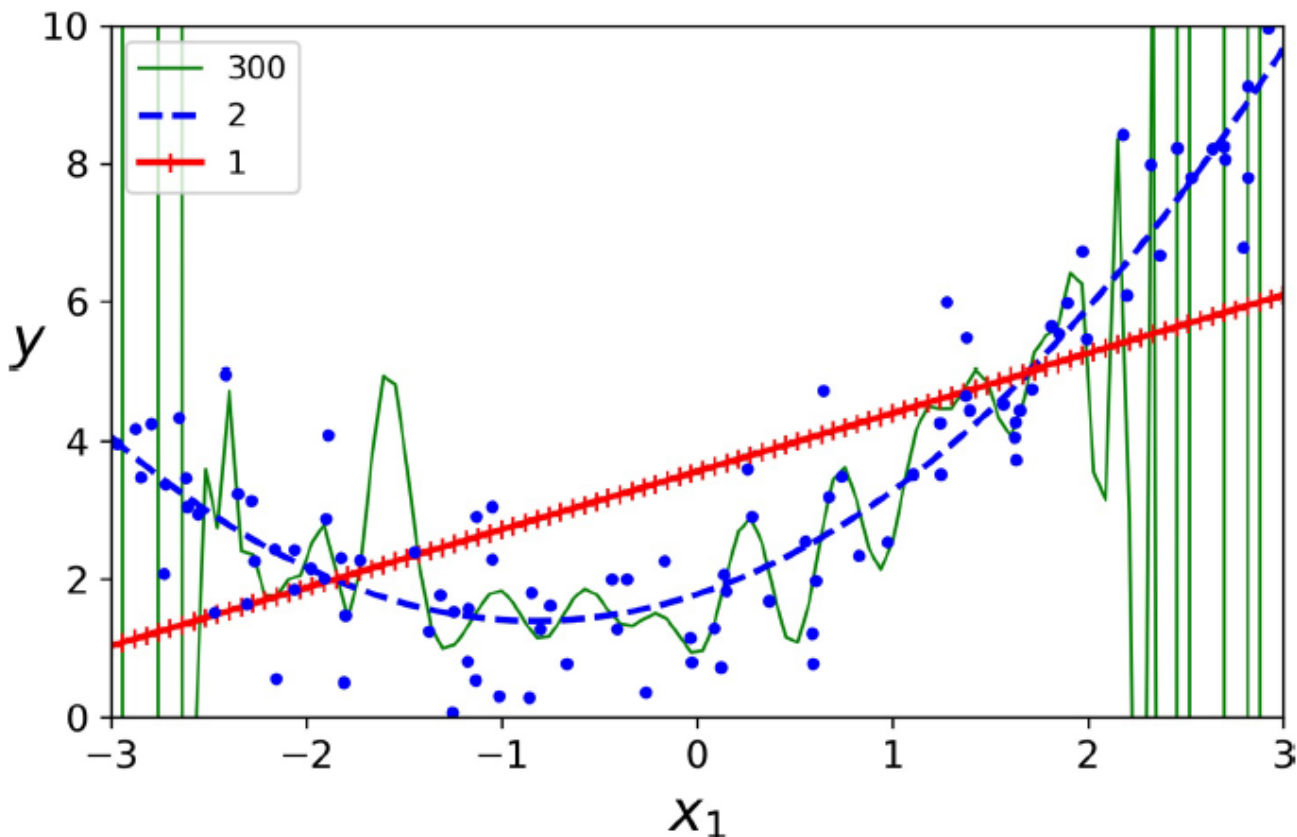


Figure 4-14. High-degree Polynomial Regression

线性模型的正则化 (regularization)

□ 正则化模型

- ✓ 对模型进行正则化，即约束模型。
- ✓ 模型拥有的自由度越少，则过拟合数据的难度就越大。
- ✓ 对线性模型，正则化通常是约束模型的权值 $\mathbf{w}$ 来实现。

□ 岭回归 (Ridge/Tikhonov/ ℓ_2 regularization, Penalty term)

$$L(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \alpha \|\mathbf{w}\|_2^2 \quad \Rightarrow \quad \hat{\mathbf{w}} = \left(\mathbf{X}^T \mathbf{X} + \alpha \mathbf{I} \right)^{-1} \mathbf{X}^T \mathbf{y}$$

- ✓ 学习算法不仅拟合数据，而且还使模型权值尽可能小。
- ✓ 训练完模型后，要使用非正则化的性能度量函数来评估模型。
- ✓ 对输入特征的缩放敏感，通常进行StandardScaler。

线性模型的正则化 (regularization)

□ 岭回归 (Ridge/Tikhonov/ ℓ_2 regularization)

$$L(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 + \alpha \|\mathbf{w}\|_2^2$$

$$\hat{\mathbf{w}} = (X^T X + \alpha I)^{-1} X^T \mathbf{y}$$

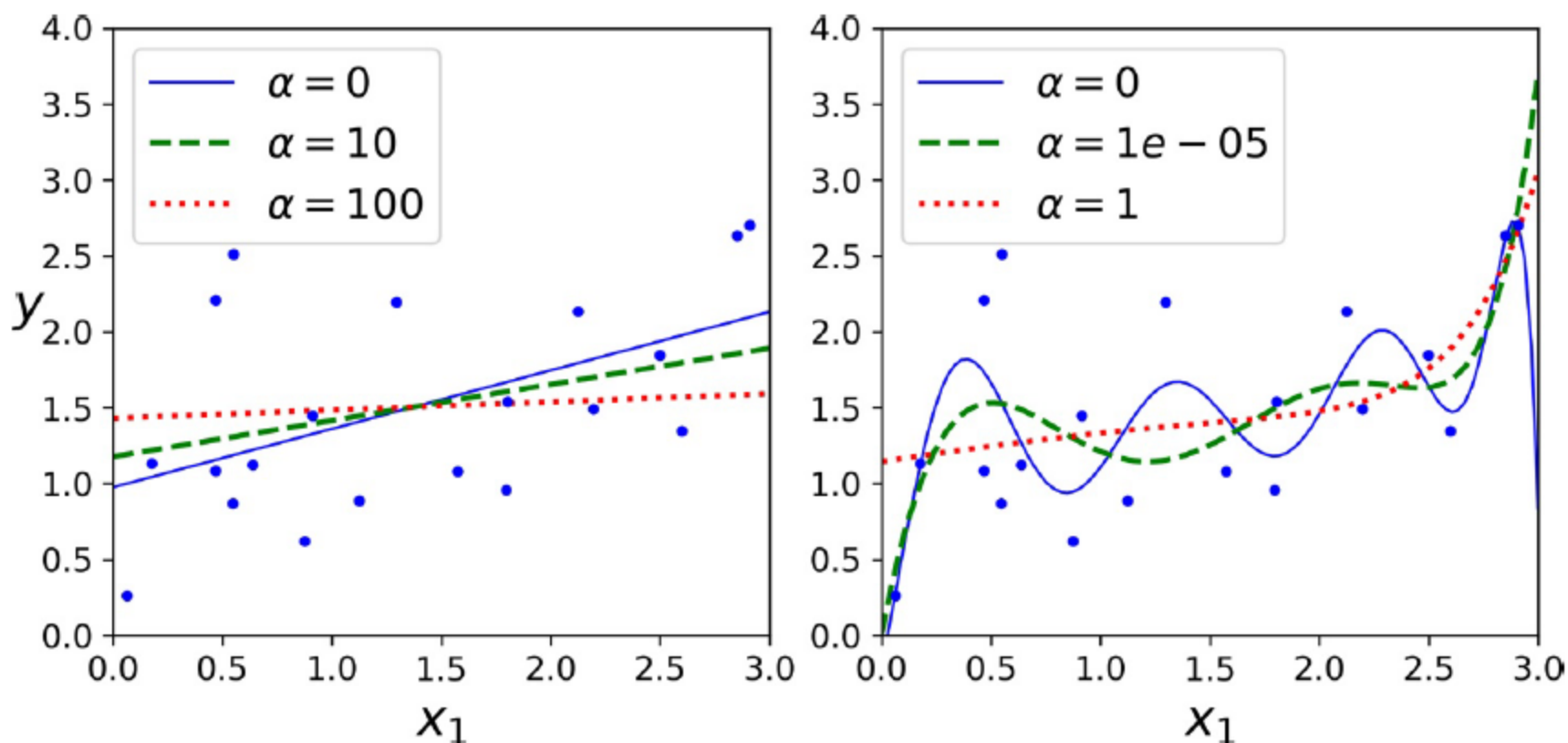


Figure 4-17. A linear model (left) and a polynomial model (right), both with various levels of Ridge regularization

线性模型的正则化 (regularization)

- Lasso回归 (ℓ_1 regularization) $L(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 + \alpha \|\mathbf{w}\|_1$
- ✓ 特点：它倾向于完全消除掉最不重要特征的权值（将它们设置为零）
- ✓ Lasso回归会自动执行特征选择并输出一个稀疏模型。

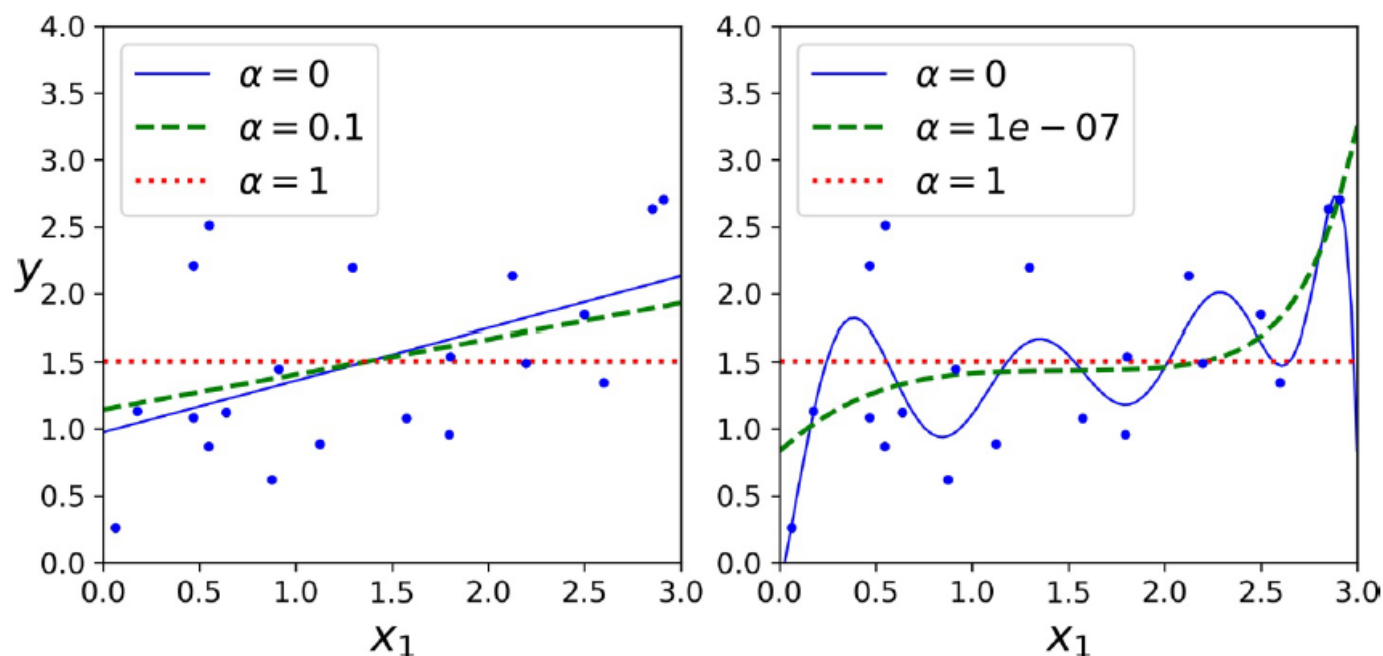


Figure 4-18. A linear model (left) and a polynomial model (right), both using various levels of Lasso regularization

线性模型的正则化 (regularization)

□ ℓ_1 vs. ℓ_2 Regularization

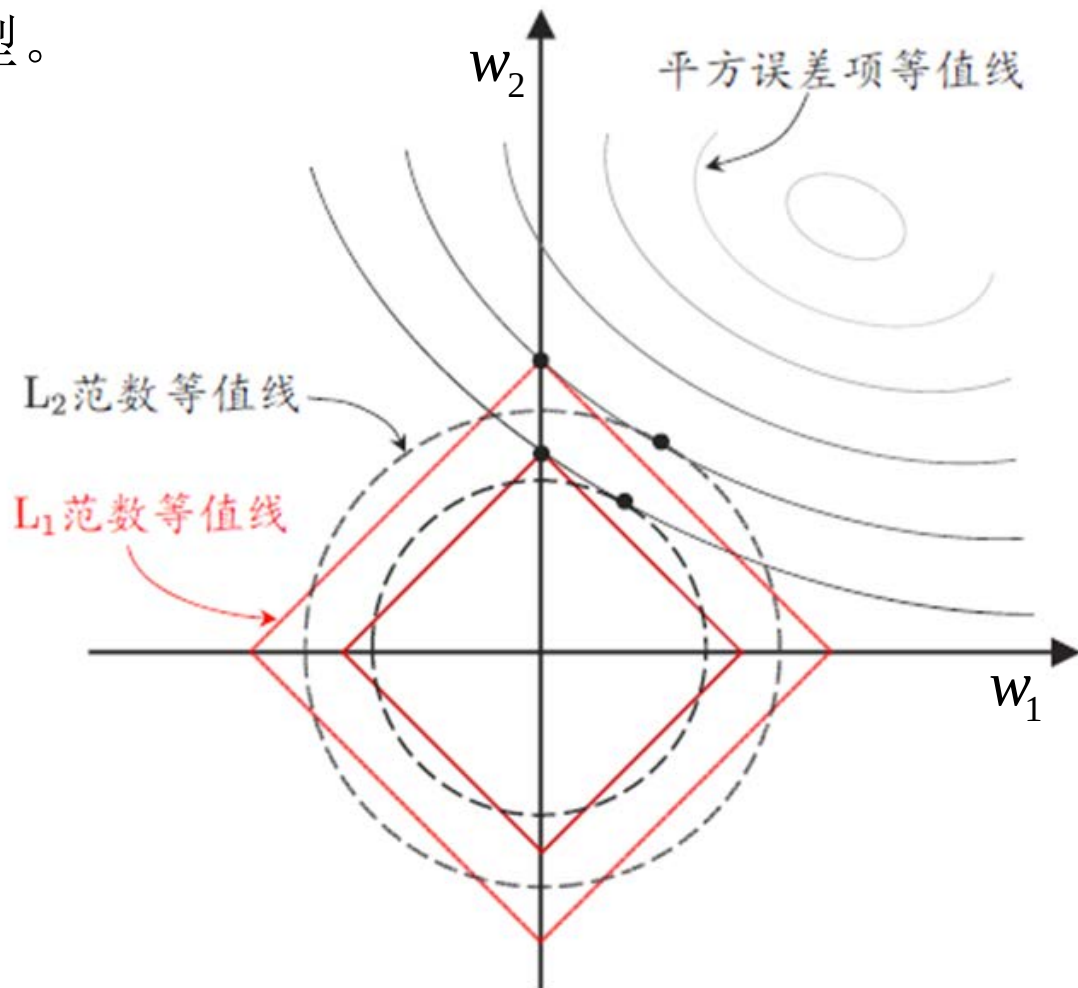
✓ Lasso回归: 稀疏模型。

l_1 -loss:

$$\|\mathbf{w}\|_1 = |w_1| + |w_2|$$

l_2 -loss:

$$\|\mathbf{w}\|_2^2 = |w_1|^2 + |w_2|^2$$



线性模型的正则化 (regularization)

□ 弹性网络 (Elastic Net)

$$L(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 + r\alpha \|\mathbf{w}\|_1 + \frac{1-r}{2} \alpha \|\mathbf{w}\|_2^2$$

□ 提前停止：适用于梯度下降算法

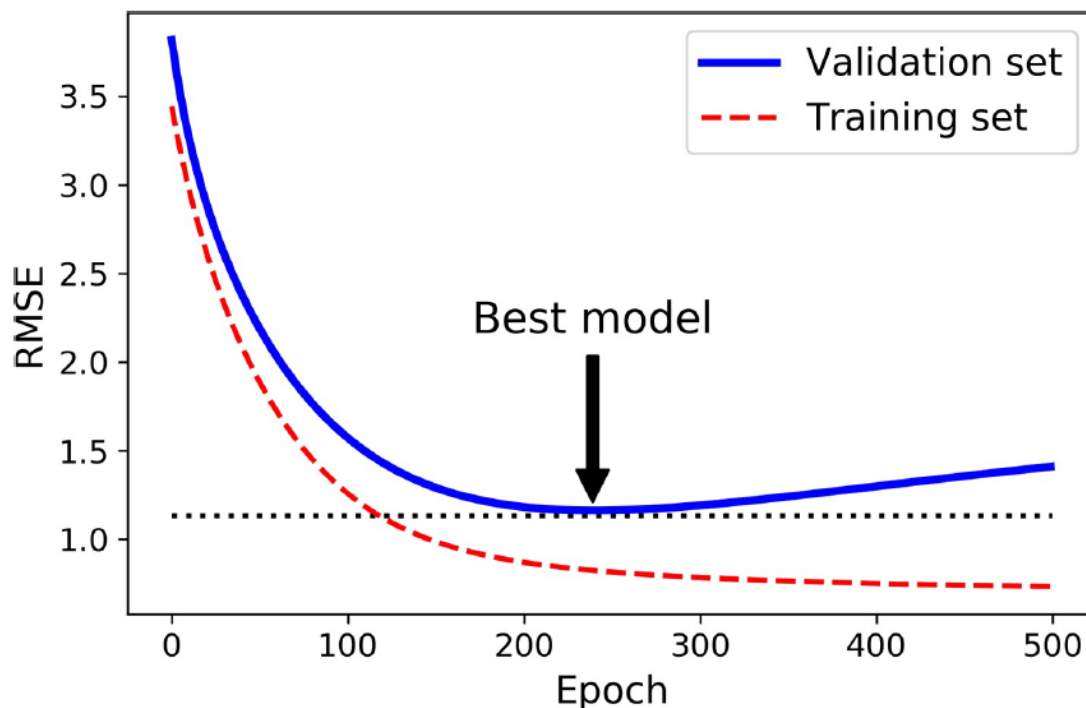


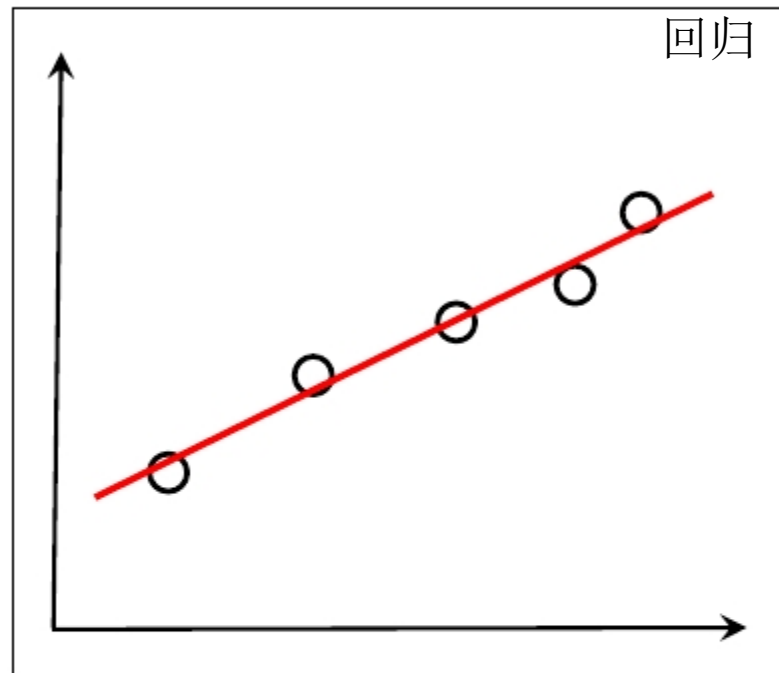
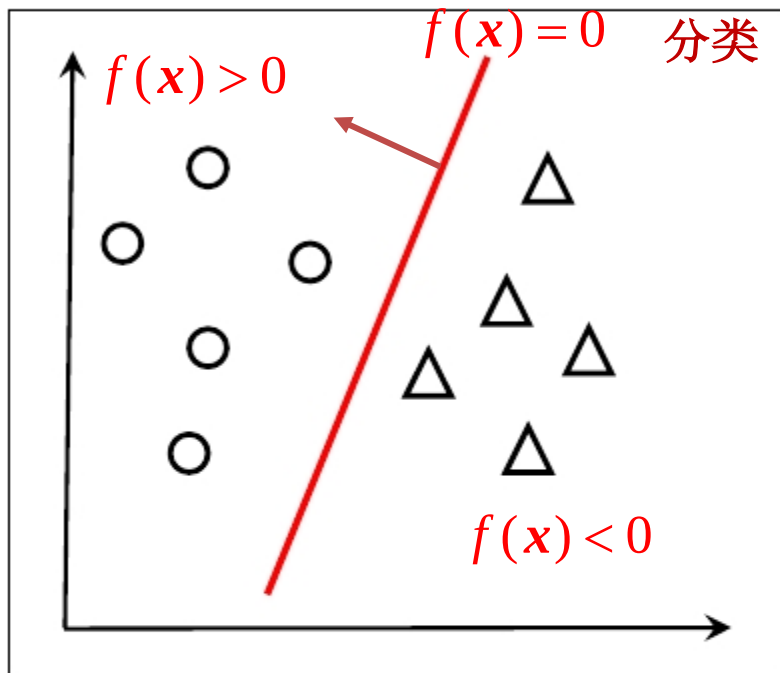
Figure 4-20. Early stopping regularization

线性模型的正则化 (regularization)

□ 判断题

L_2 正则化往往用于防止过拟合，而 L_1 正则化往往用于特征选择。

线性模型：对率回归 (Logistic Regression)



线性模型(linear model)试图学得一个通过属性的线性组合来进行预测的函数

$$f(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

$$f(x) = w^T x + b$$

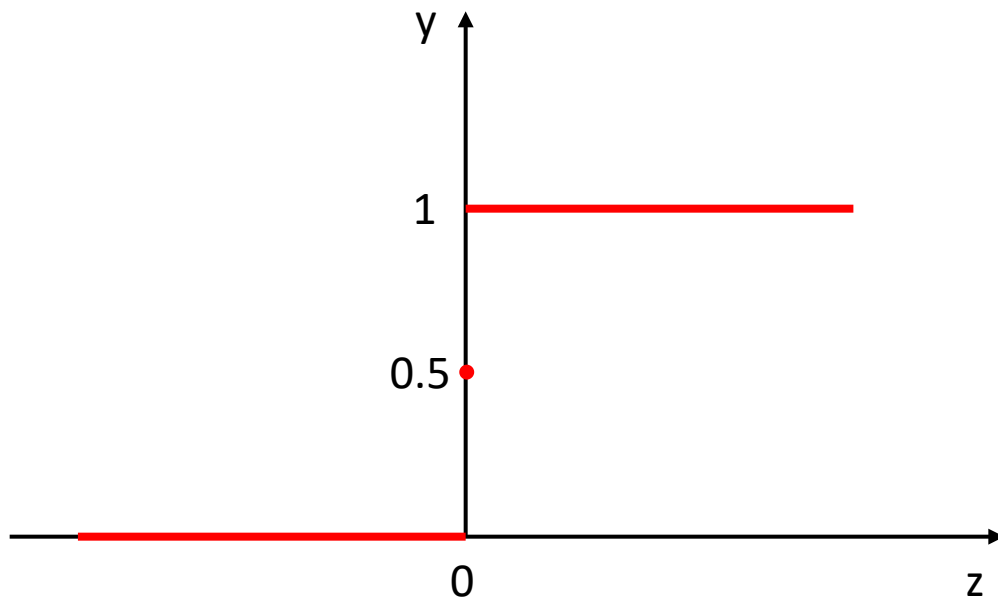
简单、基本、可理解性好

二分类任务

线性回归模型产生的实值输出 $z = \omega^T \mathbf{x} + b \in \{-\infty, +\infty\}$ 找 z 和 y 的联系函数
期望输出 $y \in \{0, 1\}$

理想的“单位阶跃函数”
(unit-step function)

$$y = \begin{cases} 0, & z < 0; \\ 0.5, & z = 0; \\ 1, & z > 0, \end{cases}$$



性质不好,
需找“替代函数”
(surrogate function)

二分类任务

线性回归模型产生的实值输出 $z = \mathbf{x}^T \boldsymbol{\omega} + b \in \underline{\{-\infty, +\infty\}}$
期望输出 $y \in \underline{\{0,1\}}$

找 z 和 y 的
联系函数

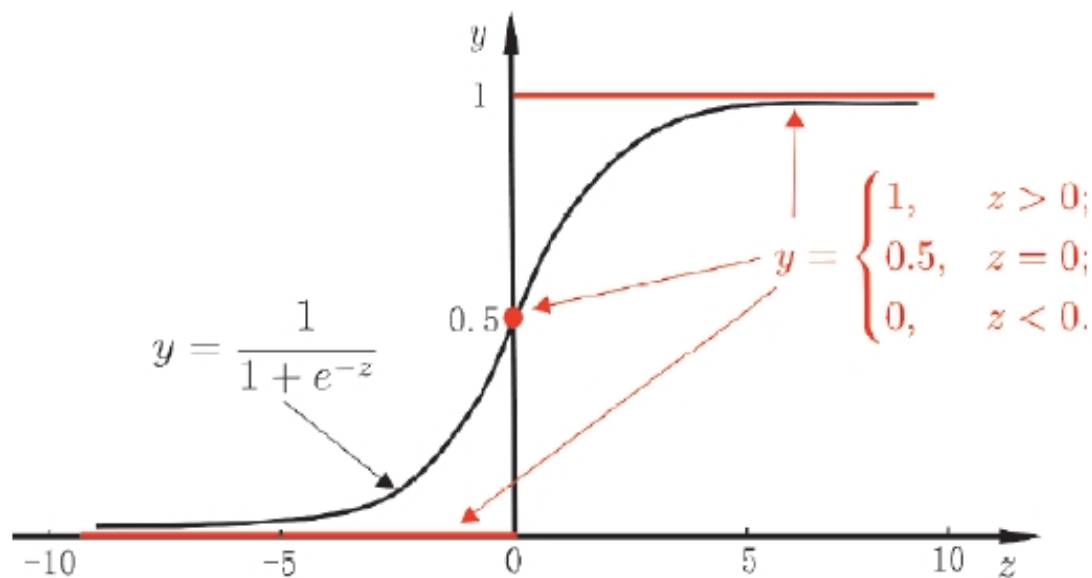
常用，单调可微、任意阶可导

(logistic function)，简称“对率函数”

即sigmoid函数

$$y = \frac{1}{1 + e^{-z}}$$

$$y' = y(1-y)$$



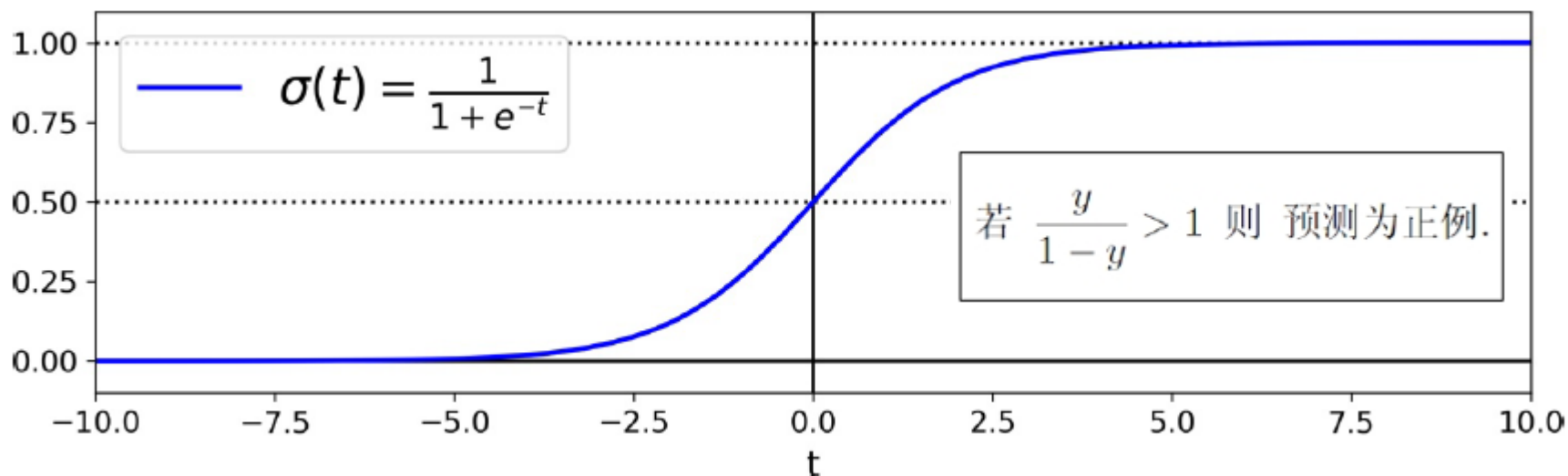
Logistic回归

□ 对率回归模型的估计概率

$$z = \mathbf{x}^T \mathbf{w} + b = \mathbf{x}^T \boldsymbol{\theta} \in (-\infty, +\infty) \quad \rightarrow \text{线性函数 (score)}$$

$$\hat{p} = \sigma(z) = \sigma(\mathbf{x}^T \boldsymbol{\theta}) \in [0, 1] \quad \rightarrow \text{非线性函数 (probability)}$$

$$\hat{y} = \begin{cases} 0, & \text{if } \hat{p} < 0.5 \\ 1, & \text{if } \hat{p} \geq 0.5 \end{cases} \quad \rightarrow \text{模型预测 (二分类)}$$



Logistic回归

以对率函数为联系函数:

$$y = \frac{1}{1 + e^{-z}} \quad \text{变为} \quad y = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

即: $\ln \frac{y}{1-y} = \mathbf{w}^T \mathbf{x} + b$

“对数几率”

(log odds, 亦称 logit) 几率(odds), 反映了 $\mathbf{x}$ 作为正例的相对可能性

$$\begin{cases} > 0, \text{即: } \frac{y}{(1-y)} > 1 \\ = 0, \text{即: } \frac{y}{(1-y)} = 1 \\ < 0, \text{即: } \frac{y}{(1-y)} < 1 \end{cases}$$

“对数几率回归” (logistic regression)
简称 “对率回归”

- 无需事先假设数据分布
- 可得到 “类别” 的近似概率预测
- 可直接应用现有数值优化算法求取最优解

注意: 它是
分类学习算法!

Logistic回归

□ 几率 (Odds)

Odds also have a simple relation with [probability](#): the odds of an outcome are the ratio of the probability that the outcome occurs to the probability that the outcome does not occur. In mathematical terms, where p is the probability of the outcome:

$$\text{odds} = \frac{p}{1 - p}$$

where $1 - p$ is the probability that the outcome does not occur.

Logistic回归

□ 训练和成本函数

- 单个训练实例的成本函数

$$\hat{c} = \begin{cases} -\log(\hat{p}), & \text{if } y=1 \\ -\log(1-\hat{p}), & \text{if } y=0 \end{cases}$$

$\hat{p}$: 将样本视为正例的概率

$1 - \hat{p}$: 将样本视为反例的概率

- 对率回归成本函数（**对数损失**）

$$L(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(\hat{p}^{(i)}) - (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

$$\frac{\partial}{\partial \theta_j} L(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \left(\sigma(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

- 无闭式解
- 是凸函数

Logistic回归



❑ 决策边界：鸢尾花数据集分类

- ✓ 该数据集包括3个鸢尾花类别，每个类别有50个样本。
- ✓ 该数据集共150行，每行1个样本。每个样本有5个字段，分别是：

- 花萼长度(单位cm)
4.4,3.2,1.3,0.2,Iris-setosa
 - 花萼宽度(单位：cm)
5.0,3.5,1.6,0.6,Iris-setosa
 - 花瓣长度(单位：cm)
5.1,3.8,1.9,0.4,Iris-setosa
 - 花瓣宽度(单位：cm)
4.8,3.0,1.4,0.3,Iris-setosa
 - 类别(共3类)
5.1,3.8,1.6,0.2,Iris-setosa
 - Iris Setosa
4.6,3.2,1.4,0.2,Iris-setosa
 - Iris Versicolour
5.3,3.7,1.5,0.2,Iris-setosa
 - Iris **Virginica**。
5.0,3.3,1.4,0.2,Iris-setosa
- 7.0,3.2,4.7,1.4,Iris-versicolor
6.4,3.2,4.5,1.5,Iris-versicolor
6.9,3.1,4.9,1.5,Iris-versicolor
5.5,2.3,4.0,1.3,Iris-versicolor
6.5,2.8,4.6,1.5,Iris-versicolor
5.7,2.8,4.5,1.3,Iris-versicolor
6.3,3.3,4.7,1.6,Iris-versicolor

Logistic回归

□ 决策边界：鸢尾花数据集分类

- 分类特征：花瓣宽度

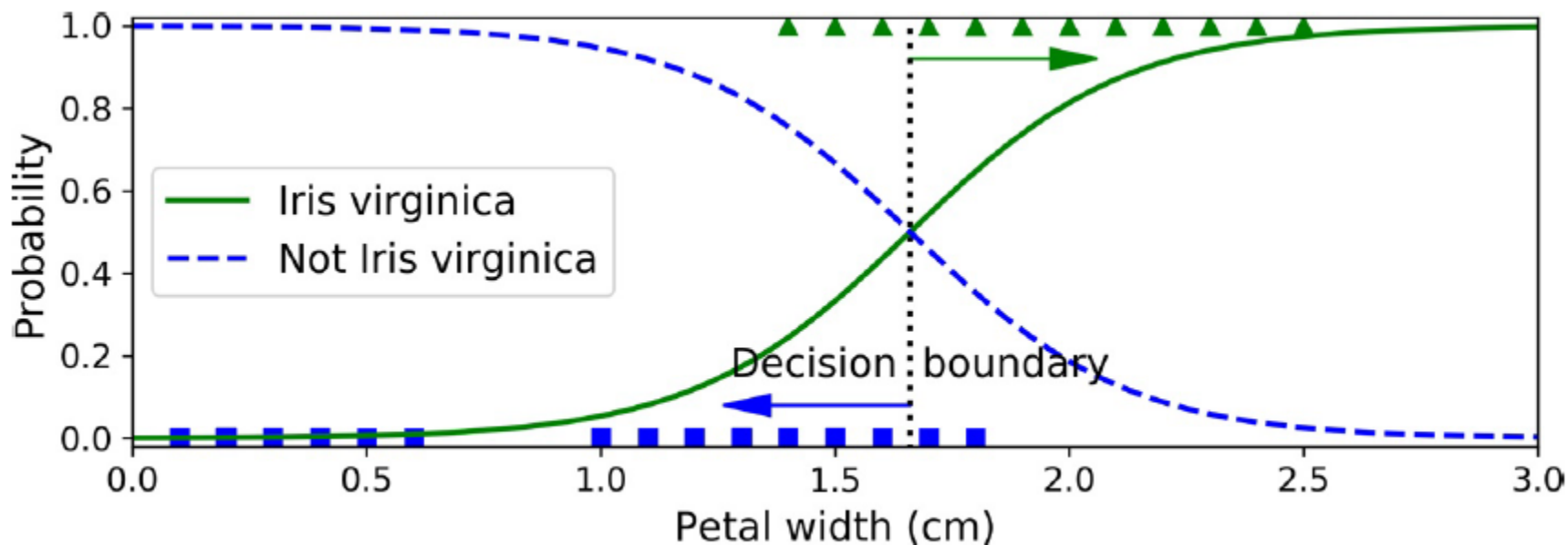


Figure 4-23. Estimated probabilities and decision boundary

维吉尼亚鸢尾花的花瓣宽度范围：1.4 ~ 2.5 cm

Logistic回归

□ 决策边界：鸢尾花数据集分类

- 分类特征：花瓣宽度，花瓣的长度
- 虚线表示模型估算概率为50%的点，即模型的决策边界。

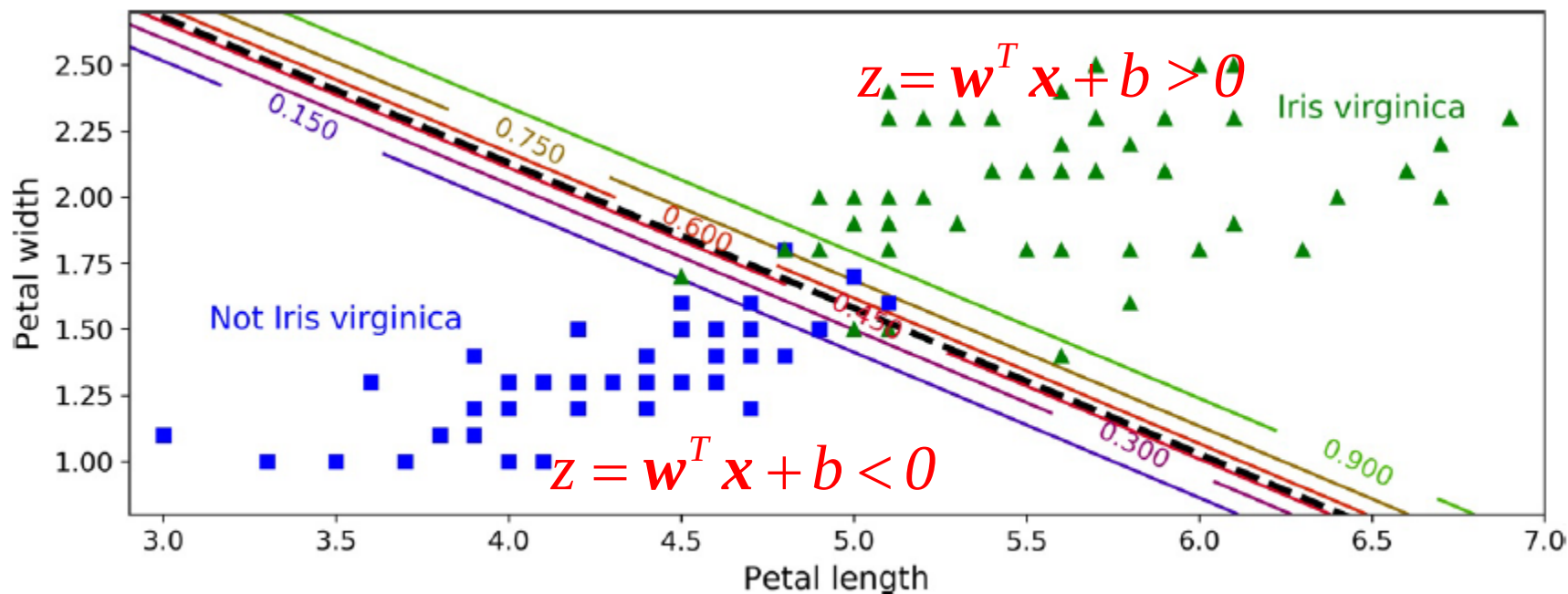


Figure 4-24. Linear decision boundary

Softmax回归（多分类任务）

□ 多元对率回归

- 类别 k 的Softmax分数（Score）定义为：

$$s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)} \in (-\infty, +\infty) \quad \rightarrow \text{线性回归函数}$$

- 注：每个类有自己的特征参数向量 $\boldsymbol{\theta}^{(k)}$

□ Softmax函数（归一化指数）

$$\hat{p} = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))} \in [0, 1]$$

□ Softmax回归分类预测

$$\hat{y} = \arg \max_k \sigma(\mathbf{s}(\mathbf{x}))_k$$

Softmax回归（多分类任务）

□ Softmax函数（归一化指数）

输出分数

$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix}$$



Softmax函数

$$\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$



概率值

$$\begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$



标签值

$$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Softmax回归（多分类任务）

□ 交叉熵损失函数（cross-entropy loss function）

$$L(\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(K)}) = \frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K -y_k^{(i)} \log(\hat{p}_k^{(i)})$$

□ 类 j 的交叉熵梯度向量

$$\nabla_{\boldsymbol{\theta}_{(j)}} L(\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(K)}) = \frac{1}{m} \sum_{i=1}^m (\hat{p}_j^{(i)} - y_j^i) \mathbf{x}^{(i)}$$

	C1	C2	C3	C4
真实分布P(Y) (OneHot编码)	1	0	0	0
预测分布P(X)	0.7	0.1	0.1	0.1

Softmax回归（多分类任务）

□ 鸢尾花数据集分类：

- Softmax三分类
- 分类特征：花瓣宽度，花瓣的长度

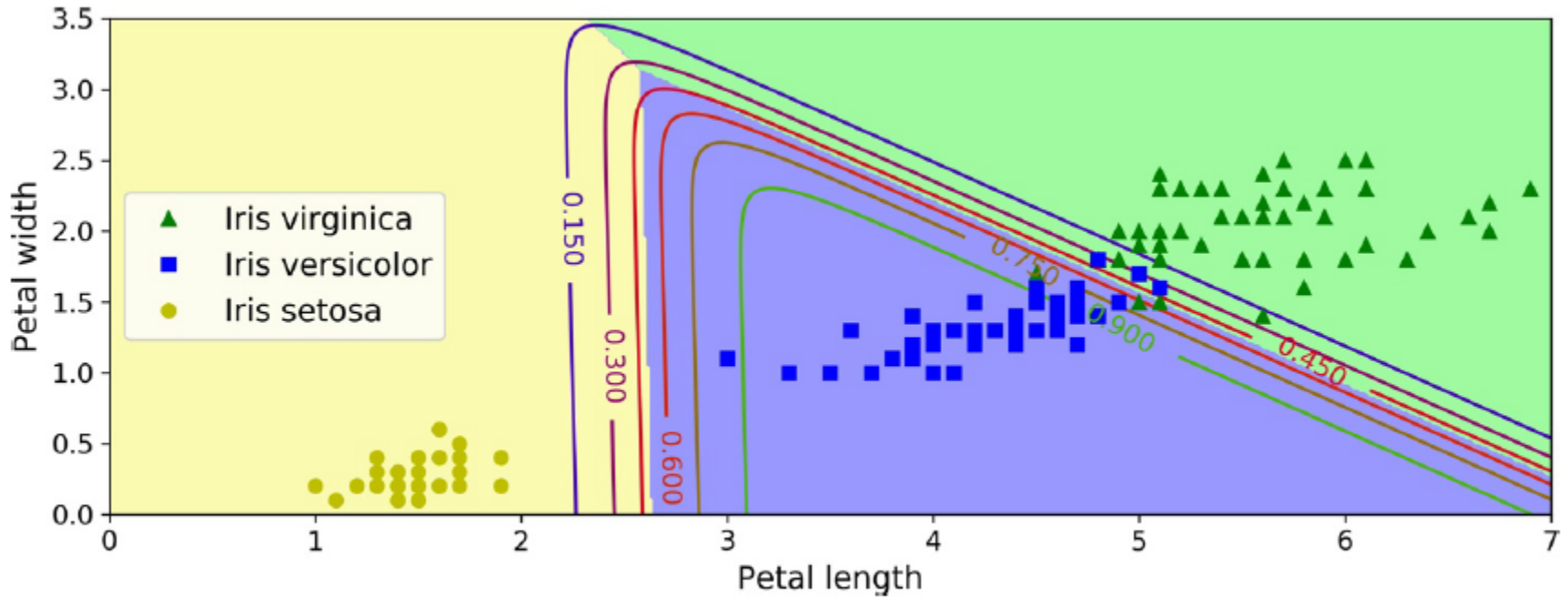


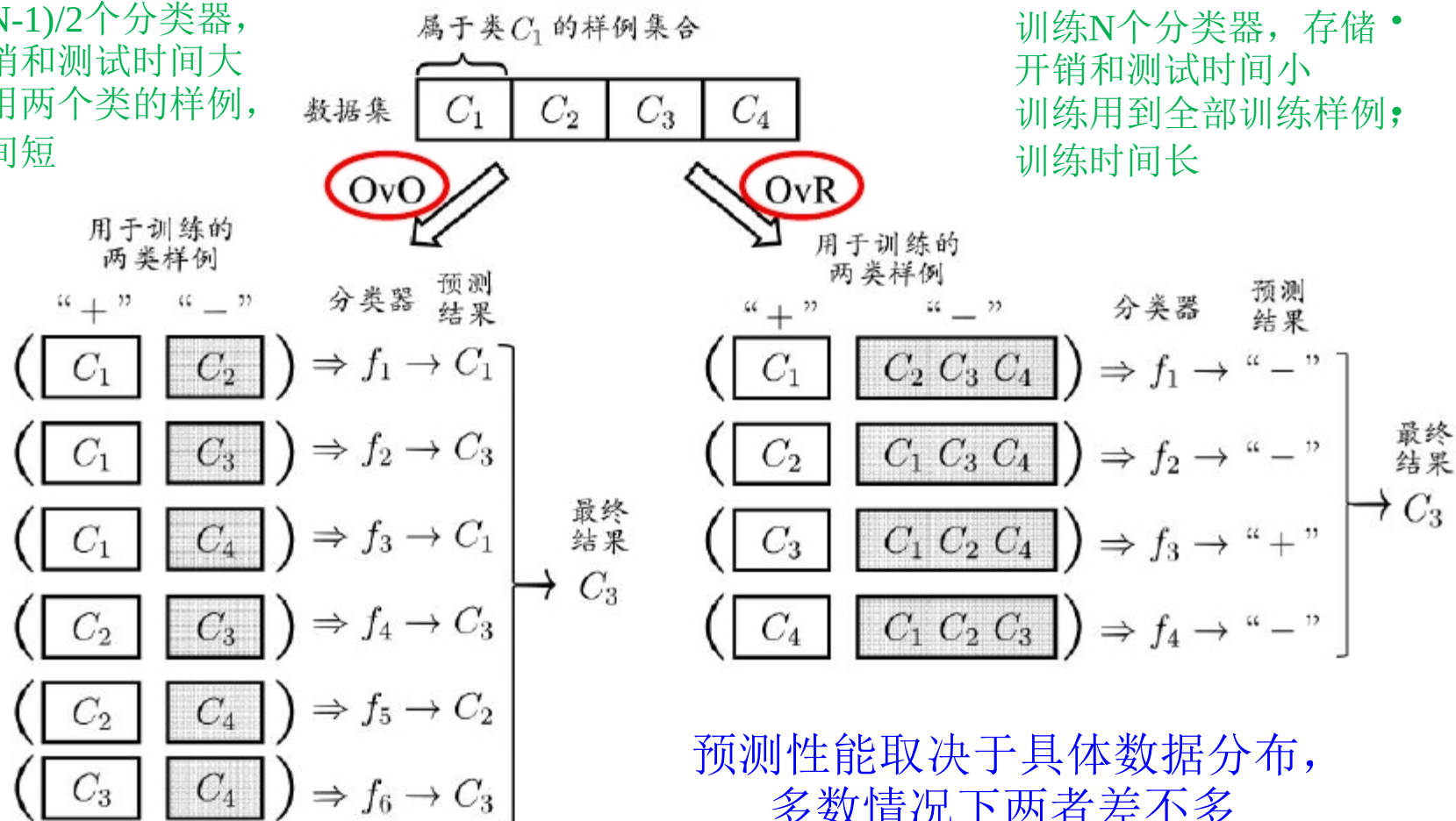
Figure 4-25. Softmax Regression decision boundaries

多分类学习

拆解法：将一个多分类任务拆分为若干个二分类任务求解

训练 $N(N-1)/2$ 个分类器，
存储开销和测试时间大
训练只用两个类的样例，
训练时间短

训练 N 个分类器，存储
开销和测试时间小
训练用到全部训练样例；
训练时间长

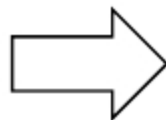


纠错输出码 (ECOC)

多对多(Many vs Many, MvM): 将若干类作为正类, 若干类作为反类

一种常见方法: 纠错输出码 (Error Correcting Output Code)

编码: 对 N 个类别做 M 次划分, 每次将一部分类别划为正类, 一部分划为反类

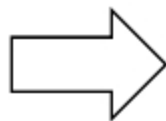


M 个二类任务;
(原)每类对应一个长为 M 的编码

距离最小的类为
最终结果



解码: 测试样本交给 M 个分类器预测



长为 M 的预测结果编码

纠错输出码

	f_1	f_2	f_3	f_4	f_5	海明距离	欧氏距离
$C_1 \rightarrow$	-1	+1	-1	+1	+1	3	$2\sqrt{3}$
$C_2 \rightarrow$	+1	-1	-1	+1	-1	4	4
$C_3 \rightarrow$	-1	+1	+1	-1	+1	1	2
$C_4 \rightarrow$	-1	-1	+1	+1	-1	2	$2\sqrt{2}$
测试示例 $\rightarrow$	-1	-1	+1	-1	+1		

(a) 二元 ECOC 码

[Dietterich and Bakiri,1995]

	f_1	f_2	f_3	f_4	f_5	f_6	f_7	海明距离	欧氏距离
$C_1 \rightarrow$	-1	-1	+1	+1	-1	+1	+1	4	4
$C_2 \rightarrow$	-1	0	0	0	+1	-1	0	2	2
$C_3 \rightarrow$	+1	+1	-1	-1	-1	+1	-1	5	$2\sqrt{5}$
$C_4 \rightarrow$	-1	+1	0	+1	-1	0	+1	3	$\sqrt{10}$
测试示例 $\rightarrow$	-1	+1	+1	-1	+1	-1	+1		

(b) 三元 ECOC 码

[Allwein et al. 2000]

- ECOC编码对分类器错误有一定容忍和修正能力，编码越长、纠错能力越强
- 对同等长度的编码，理论上来说，任意两个类别之间的编码距离越远，则纠错能力越强

类别不平衡 (class-imbalance)

不同类别的样本比例相差很大；“小类”往往更重要

基本思路：

若 $\frac{y}{1-y} > 1$ 则 预测为正例.



若 $\frac{y}{1-y} > \frac{m^+}{m^-}$ 则 预测为正例.

基本策略

—— “再缩放” (rescaling):

$$\frac{y'}{1-y'} = \frac{y}{1-y} \times \frac{m^-}{m^+}$$

然而，精确估计 m^-/m^+ 通常很困难！

常见类别不平衡学习方法：

- 过采样 (oversampling)

例如：SMOTE

- 欠采样 (undersampling)

例如：EasyEnsemble

- 阈值移动 (threshold-moving)

本章小结

□ 线性回归

- ✓ 线性回归
 - 闭式解（最小二乘法）
 - 梯度下降法
- ✓ 多项式回归
- ✓ 模型的正则化

□ 线性分类

- ✓ 二分类：Logistic回归
- ✓ 多分类：
 - Softmax回归
 - OvO、OvR、MvM

本章作业

□ 课后实践内容：

- ✓ 《机器学习实战》第4章。
- ✓ 《动手学深度学习》第3章。

□ 本章作业：

- 推导岭回归模型的梯度向量、闭式解。
- 推导Lasso回归模型的梯度梯度、闭式解。
- 推导Logistic、Softmax分类的交叉熵梯度向量。
- 设有一目标函数 $J(\Theta) = \theta_1^2 + \theta_2^2 + 3$ ，用梯度下降法求解目标函数的解。要求：（1）写出梯度下降法求解的基本流程图；（2）求解迭代3次后，参数向量 $\Theta = (\theta_1, \theta_2)$ 的值（假定初始的起点为 $\Theta^{(0)} = (1, 2)$ ，学习率 $\eta = 0.1$ ，写出每次迭代的计算过程）。

前往第四站.....

